ESTIMATIVA E MÉTRICAS SOFTWARE

AULA 5

Prof.ª Rosemari Pavan Rattmann

CONVERSA INICIAL

Nesta abordagem, vamos discutir a importância das métricas para o desenvolvimento e a manutenção de aplicações.

Começaremos analisando as métricas para aplicações web e aplicativos móveis, onde abordaremos desempenho, segurança e usabilidade. Esses indicadores são essenciais para garantir que nossos produtos atendam às expectativas dos usuários e operem de forma eficaz.

Em seguida, nos aprofundaremos nas métricas de código-fonte, explorando como a qualidade do código impacta o desenvolvimento. Focaremos em aspectos como cobertura de código, densidade de defeitos e o tempo de teste, além de discutirmos a importância do rastreamento de requisitos e suas aplicações práticas em ambientes industriais.

Depois, falaremos sobre as métricas para projetos de manutenção e melhorias. Vamos introduzir o conceito de manutenibilidade e as métricas que podem ser utilizadas para garantir a qualidade durante a manutenção, alinhando-nos a padrões como a ISO, além de trazer exemplos práticos.

Por fim, conheceremos ferramentas e repositórios de métricas, como SonarQube, JUnit, JMeter, Postman e Checkmarx. Essas ferramentas nos ajudarão a coletar e analisar dados de forma eficiente, permitindo uma gestão mais eficaz dos nossos projetos.

Preparem-se para uma discussão rica e interativa sobre como as métricas podem elevar a qualidade e a eficiência dos nossos trabalhos em software. Vamos começar!

TEMA 1 – MÉTRICAS PARA WEBAPPS E APLICATIVOS MÓVEIS

Um dos principais desafios enfrentados no desenvolvimento de software é a diversidade de plataformas e dispositivos disponíveis. Essa variabilidade torna a padronização das métricas de qualidade uma tarefa complexa. A falta de um conjunto unificado de métricas pode dificultar a avaliação consistente da qualidade dos aplicativos, levando a resultados imprecisos que podem comprometer a experiência do usuário.

Aplicativos web e móveis possuem diferentes demandas quando o assunto é desempenho e qualidade, refletindo as particularidades de cada plataforma. No caso dos aplicativos web, acessados por meio de navegadores,

é essencial que a infraestrutura seja capaz de oferecer uma experiência fluida e rápida para o usuário, independentemente das variações de dispositivos e condições de rede. Indicadores como o tempo de carregamento, a latência e a escalabilidade são fundamentais para garantir que o sistema suporte um grande número de acessos simultâneos sem comprometer a performance.

Por outro lado, os aplicativos móveis, desenvolvidos para operar em sistemas operacionais específicos, têm um acesso mais direto aos recursos do dispositivo e, por isso, a avaliação de fatores como o consumo de bateria, o uso de memória, a estabilidade e a capacidade de funcionar offline se torna indispensável. Esses elementos impactam diretamente a experiência do usuário, uma vez que determinam tanto a durabilidade do dispositivo quanto a confiabilidade do aplicativo.

Monitorar métricas específicas para cada tipo de plataforma é, portanto, um passo importante para garantir que o aplicativo atenda às expectativas dos usuários e se mantenha competitivo. Essas medições ajudam a identificar pontos de melhoria e assegurar que o desempenho do software esteja alinhado com as necessidades e o contexto de uso do público-alvo.

1.1 Métricas para aplicativos móveis

As métricas de qualidade são uma ferramenta indispensável para avaliar e aperfeiçoar o desempenho e a funcionalidade de aplicativos móveis, garantindo que ofereçam uma experiência completa e eficiente aos usuários. Na era digital atual, em que a demanda por aplicações móveis cresce a cada dia, esses critérios de avaliação se tornam ainda mais críticos, uma vez que o mercado exige não apenas alta performance, mas também compatibilidade com uma ampla gama de dispositivos, cada um com especificações e condições de uso distintas. A mensuração dessas métricas permite identificar pontos fracos e áreas de aprimoramento de maneira concreta e precisa, embasando o processo de desenvolvimento em dados técnicos que asseguram o atendimento às necessidades reais dos usuários (Kappel). A Figura 1 mostra alguns indicadores que podem ser utilizados para tomada de decisões quanto ao desenvolvimento de aplicativos móveis.

Figura 1 – Comparativo de indicadores de qualidade de aplicativos móveis

Indicador de Qualidade	APP A	APP B	APP C	Conclusão
Tempo de Resposta (seg)	0,8s (Excelente)	1,2s (Bom)	0,6s (Excelente)	Usuários preferem apps com tempos de resposta rápidos
Tempo de Carregamento (seg)	2s (Aceitável)	1,5s (Muito Bom)	1,8s (Bom)	Tempo de carregamento impacta menos que resposta rápida
Consumo de Memória (Mb)	250Mb (Médio)	200Mb (Baixo)	280Mb (Alto)	Consumo de memória moderado é preferido
Consumo de Bateria (%/hora)	6% (Moderado)	5% (Baixo)	4% (Muito Baixo)	Baixo consumo de bateria é uma preferência significativa
Eficiência de Processamento	1s (Bom)	0,9s (Muito Bom)	1,1s (Bom)	Melhoria marginalmente notada
Taxa de Erros/ Travamentos	3/1000 sessões (Moderado)	1/1000 sessões (Baixo)	2/1000 sessões (Baixo)	Alta escalabilidade aumenta preferência
Satisfação com Usabilidade	75% (Boa)	85% (Excelente)	78% (Muito Boa)	Usabilidade fortemente correlacionada com preferência

Entre os principais indicadores de qualidade, o desempenho se destaca como uma métrica central. Esse parâmetro engloba a análise de aspectos como o tempo de resposta do aplicativo, o consumo de memória e bateria, e a eficiência no processamento de dados. Com esses dados em mãos, os desenvolvedores podem identificar gargalos que podem comprometer a experiência de uso, além de detectar o impacto da aplicação no dispositivo. Isso é especialmente importante no caso de aplicativos móveis, que podem ser utilizados em cenários diversos de conectividade e em aparelhos com configurações de hardware variadas. Ao otimizar o uso de recursos, evita-se que o aplicativo trave, opere com lentidão ou apresente falhas de compatibilidade, o

que se traduz em uma experiência de uso mais consistente e agradável para o usuário final.

Outro aspecto fundamental para assegurar a qualidade de um aplicativo móvel é a métrica de funcionalidade, que se refere ao cumprimento adequado dos requisitos operacionais e à capacidade do aplicativo de operar de maneira estável em diferentes situações. Essa métrica inclui a avaliação de integrações com outros aplicativos e funcionalidades essenciais do dispositivo, como GPS, câmera e sensores. Além disso, verifica-se se as operações básicas, como login, navegação e as principais interações do usuário com a interface, são concluídas sem erros ou travamentos. Esse tipo de teste é muito relevante para que o aplicativo apresente um desempenho confiável, permitindo que o usuário realize suas atividades sem interrupções, independentemente do ambiente em que o dispositivo esteja inserido. A funcionalidade assegura que o sistema cumpre o que promete e consegue responder às necessidades dos usuários de forma eficaz, sem comprometer a qualidade da experiência.

A usabilidade também desempenha um papel essencial na experiência de uso de um aplicativo móvel, pois considera a facilidade de uso e a qualidade da interação do usuário com a interface. Essa métrica avalia, por exemplo, o quão intuitiva é a navegação, a disposição dos elementos visuais e o quanto o usuário consegue operar o aplicativo sem dificuldades. Aplicativos que possuem uma usabilidade bem planejada tendem a ser mais bem recebidos, pois proporcionam uma experiência fluida e sem barreiras. Ao mensurar a usabilidade, a equipe de desenvolvimento consegue obter dados valiosos sobre os pontos que causam frustração e desconforto, permitindo melhorias que transformem o aplicativo em um ambiente mais intuitivo e eficiente para o usuário. Uma interface de fácil compreensão e uso agrega valor ao aplicativo e contribui para que o usuário se sinta satisfeito e confortável ao utilizá-lo.

As métricas de segurança ocupam uma posição de destaque, especialmente em tempos de crescente preocupação com a privacidade e a proteção de dados. A segurança de um aplicativo não se limita a proteger informações básicas de login, mas envolve também a garantia de que os dados sensíveis e pessoais dos usuários estão protegidos contra acessos não autorizados. Essa métrica permite que o aplicativo esteja preparado para responder a possíveis ataques e que possua mecanismos de autenticação robustos, controle de acesso e práticas de codificação seguras. Além disso, a

segurança inclui a proteção contra vulnerabilidades como injeção de código e falhas que possam ser exploradas por atacantes. Em um cenário onde a segurança digital é cada vez mais relevante, essas práticas asseguram que o aplicativo seja confiável e que resguarde os dados dos usuários de maneira responsável.

Essas métricas de desempenho, funcionalidade, usabilidade e segurança são, em conjunto, a base de uma análise de qualidade abrangente para qualquer aplicativo móvel. Cada métrica oferece informações estratégicas que permitem à equipe de desenvolvimento aperfeiçoar o produto, garantindo que ele seja bem-sucedido ao atender às expectativas e necessidades dos usuários. É fundamental que os desenvolvedores adotem uma abordagem sistemática para a coleta e análise dessas métricas. A partir de uma avaliação criteriosa de todos esses fatores, é possível assegurar que o aplicativo, além de funcional, seja intuitivo, seguro e de alta performance, resultando em uma experiência satisfatória e diferenciada para o usuário final (Nayebi).

A Figura 2 apresenta alguns exemplos de indicadores sobre a escolha entre aplicações web e móvel, considerando o propósito da aplicação e o contexto de uso, visto que cada um tem vantagens específicas para determinados tipos de usuários e necessidades.

Figura 2 – Comparativo entre apps móveis e apps web

Indicador de Qualidade	Aplicativos WEB	Aplicativos Móveis
Tempo de Carregamento	Depende da infraestrutura e da rede, com alta variação; carrega por meio de navegador, podendo ter impacto na experiência	Geralmente mais rápido, após instalação, pois utiliza recursos locais e cache de dados
Latência	Pode ser elevada dependendo da conexão e da carga no servidor, impacta na escalabilidade	Latência geralmente menor, mas pode variar conforme o uso de APIs e conexão
Escalabilidade	Escalável para milhares de usuários simultâneos com uma infraestrutura de servidor robusta	Escalabilidade limitada ao dispositivo, mas pode expandir via cloud para funções online

Consumo de Bateria	Geralmente baixo, pois executa no navegador e não utiliza recursos intensivos	Alto em casos de uso intensivos de recursos (GPS, câmera etc.), impactando a experiência
Uso de Memória	Moderado e controlado pelo navegador; depende da complexidade da aplicação	Pode ser alto, especialmente para apps gráficos e que demandam recursos intensivos
Estabilidade	Alta escalabilidade se a infraestrutura é bem gerida; problemas podem ocorrer em redes lentas	Variável, dependente do sistema operacional e da compatibilidade com o dispositivo
Capacidade Offline	Limitada, geralmente necessitando conexão para carregamento completo e funcionalidade	Frequentemente oferece operação offline com funcionalidades reduzidas ou sincronização posterior
Experiência do Usuário (UX)	Depende do navegador e do design responsivo para diversos dispositivos e tamanhos de tela	Interface otimizada para o dispositivo; pode integrar funcionalidades específicas (notificações push, etc)

1.2 Métricas para aplicações web

A qualidade das aplicações web é um tema fundamental no desenvolvimento de software, especialmente em um cenário onde a presença online é determinante para o sucesso de empresas e organizações. Entre os aspectos mais importantes a serem considerados estão o tempo de resposta, o tempo de carregamento e a acessibilidade. Cada um desses elementos desempenha um papel significativo na experiência do usuário e, consequentemente, na aceitação e utilização das aplicações (Knott).

O tempo de resposta é uma métrica importante, pois se refere ao intervalo entre a solicitação do usuário e o momento em que a aplicação começa a processar essa solicitação. Um tempo de resposta adequado é imprescindível para manter o usuário engajado; se esse tempo for longo, a frustração pode levar à perda de interesse e ao abandono da aplicação. Portanto, é fundamental que os desenvolvedores otimizem a eficiência do código e a configuração do servidor, além de garantir que os recursos utilizados na aplicação sejam geridos de forma eficaz. Uma resposta rápida não apenas melhora a satisfação do usuário, mas também contribui para uma percepção positiva da marca.

O tempo de carregamento é outra métrica vital que determina quanto tempo leva para que uma página web esteja completamente acessível ao usuário. Assim como o tempo de resposta, um tempo de carregamento reduzido é determinante para uma experiência de navegação fluida. Esse aspecto é tão importante que também afeta o posicionamento nos motores de busca, já que sites mais rápidos tendem a ser priorizados em relação aos mais lentos. Assim, otimizar o tempo de carregamento deve ser uma prioridade no desenvolvimento de qualquer aplicação web, garantindo que os usuários possam acessar o conteúdo desejado rapidamente e sem dificuldades.

A acessibilidade é um aspecto frequentemente subestimado, mas essencial na criação de aplicações web. Garantir que todos, incluindo pessoas com deficiências, possam navegar e interagir com o aplicativo é um requisito não apenas ético, mas muitas vezes legal. Isso implica em adotar práticas de design que considerem a diversidade de usuários e suas necessidades específicas, utilizando recursos que permitam a interação por meio de dispositivos assistivos. Ao promover a acessibilidade, os desenvolvedores ampliam o alcance da aplicação, permitindo que um número maior de pessoas possa utilizá-la de maneira efetiva.

As métricas de qualidade devem ser monitoradas continuamente ao longo do ciclo de vida da aplicação, e a coleta e análise regular desses dados permitem que os desenvolvedores identifiquem problemas rapidamente e façam ajustes proativos, garantindo que a aplicação se mantenha em conformidade com os padrões de qualidade esperados. Essa abordagem não apenas melhora a qualidade do software, mas também assegura que as equipes de desenvolvimento estejam sempre prontas para se adaptar a mudanças nas expectativas dos usuários e nas novas tecnologias emergentes.

A interconexão entre diferentes métricas de qualidade não pode ser negligenciada. Melhorias em uma área, como o tempo de resposta, podem ter um impacto positivo em outras métricas, como o tempo de carregamento e a acessibilidade. Portanto, é fundamental que os desenvolvedores adotem uma abordagem integrada, considerando as métricas de forma holística. Isso significa que, ao planejar e executar testes, é preciso levar em conta como cada aspecto da aplicação pode afetar os demais (Knott).

A busca por alta qualidade nas aplicações web deve ser uma prioridade constante para os desenvolvedores. Ao focar em métricas como tempo de

resposta, tempo de carregamento e acessibilidade, e ao garantir que essas métricas sejam monitoradas e melhoradas continuamente, é possível criar aplicações que atendam às necessidades dos usuários de forma eficaz. Dessa maneira, os desenvolvedores não apenas asseguram uma experiência satisfatória, mas também estabelecem um diferencial competitivo em um mercado cada vez mais dinâmico e exigente.

Para garantir uma aplicação de alta qualidade, é fundamental que as métricas sejam monitoradas continuamente ao longo do ciclo de vida do software, seja um aplicativo móvel ou uma aplicação web. Essa vigilância permite que os desenvolvedores identifiquem e solucionem problemas rapidamente, ajustando a aplicação conforme necessário e gerando novas versões, muitas vezes antecipando reclamações dos usuários. Isso pode melhorar a experiência do usuário e manter a aplicação competitiva em um mercado em rápida evolução. A busca pela qualidade deve ser uma prioridade contínua, assegurando que as aplicações atendam às necessidades dos usuários e superem suas expectativas (Bachmann).

Vamos analisar com mais detalhes os diversos tipos de métricas.

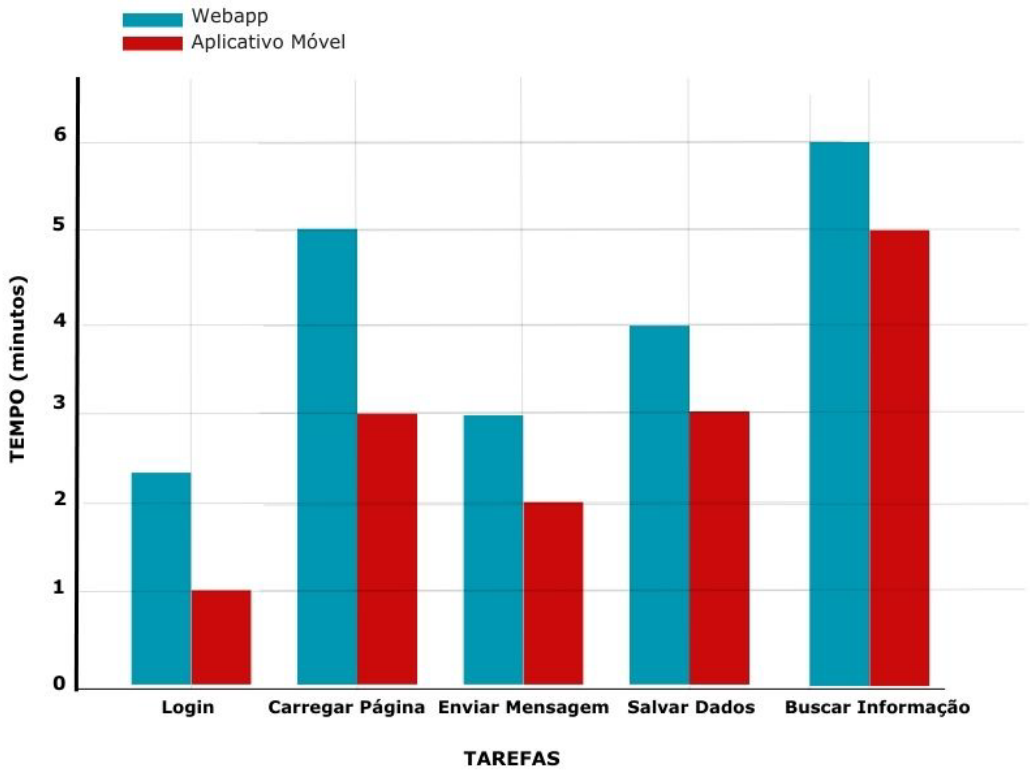
1.3 Métricas de desempenho

São essenciais para verificar a rapidez e a eficiência da aplicação. Elas medem o tempo de resposta das ações, a velocidade de carregamento de conteúdo e o consumo de recursos, como memória e bateria. Com essas métricas, é possível detectar gargalos e garantir que o aplicativo funcione bem mesmo em diferentes dispositivos e condições de rede, proporcionando uma experiência fluida, ou seja, a sensação de continuidade e naturalidade que o usuário tem ao interagir com a aplicação. O desempenho, como um aspecto crítico na engenharia de aplicações web, foca em métricas que garantem a eficiência e a responsividade. As principais métricas incluem:

- Tempo de resposta: mede o tempo que o sistema leva para responder a uma solicitação do usuário. A Figura 3 apresenta uma comparação de tempos para completar tarefas.
- Taxa de transferência: quantidade de dados processados pelo sistema em um período, essencial para aplicações com muitos dados.
- Escalabilidade: capacidade de manter o desempenho ao aumentar o número de usuários ou a complexidade dos dados.

- Confiabilidade de resposta: analisa a frequência de falhas, interrupções ou latências.

Figura 3 – Métrica de tempo de resposta



1.4 Métricas de segurança

São fundamentais para assegurar a proteção dos dados e a privacidade do usuário. Elas medem a eficácia de métodos de autenticação, proteção contra ataques (como SQL injection ou XSS) e a integridade dos dados armazenados. Com a crescente preocupação com a privacidade digital, essas métricas garantem que o aplicativo esteja preparado para prevenir e lidar com possíveis ameaças de forma eficaz. A segurança é abordada como um requisito essencial, com métricas que verificam a integridade, a confidencialidade e a disponibilidade. As principais métricas são:

- Proteção de dados: mede o nível de encriptação e métodos de autenticação para proteger os dados dos usuários.
- Resiliência contra ataques: capacidade do sistema de resistir a ataques como injeção de SQL, cross-site scripting (XSS) e cross-site request forgery (CSRF).

- Controle de acessos: quantifica a eficácia dos mecanismos de controle de acessos, como autorização e autenticação.
- Auditoria de logs: verifica a capacidade de rastrear e auditar ações para detectar e responder a incidentes.

1.5 Métricas de usabilidade

Avaliam a facilidade com que o usuário consegue navegar e interagir com a aplicação. Isso inclui medir a clareza da interface, a quantidade de passos necessários para completar uma tarefa e a taxa de erro dos usuários. Essas métricas ajudam a identificar pontos de frustração ou dificuldade, permitindo ajustes no design para tornar o aplicativo mais intuitivo e agradável de usar. A usabilidade é tratada como um fator que afeta diretamente a experiência do usuário e a satisfação. As principais métricas de usabilidade incluem:

- Facilidade de navegação: avalia o design e a arquitetura da informação para garantir que os usuários possam navegar facilmente. A facilidade de navegação é um dos principais fatores que afetam a usabilidade de um aplicativo ou site. Essa métrica se refere à clareza e à simplicidade com que um usuário pode se mover por diferentes seções da aplicação. Uma navegação intuitiva permite que os usuários encontrem as informações que buscam rapidamente, sem frustrações. Para garantir essa facilidade, é crucial que o design da interface seja coeso, com uma arquitetura de informação bem estruturada. Menus claros, links bem definidos e uma organização lógica das páginas ajudam a criar um fluxo natural que guia os usuários, permitindo que eles interajam com a aplicação de maneira fluida e agradável. Aplicativos que falham em oferecer uma navegação adequada podem levar à insatisfação e ao abandono do uso, tornando essa métrica essencial para o sucesso do projeto (Nayebi).
- Eficiência de interação: mede o número de cliques, toques e passos necessários para completar tarefas comuns e o quanto os usuários conseguem realizar suas tarefas em um aplicativo ou site de forma rápida e com o mínimo de esforço. Essa métrica pode ser avaliada observando-se o tempo necessário para completar uma tarefa específica, como preencher um formulário ou realizar uma compra. Quanto menos tempo e cliques um usuário precisa para alcançar seu objetivo, maior é a eficiência. Para aumentar essa eficiência, é fundamental que os desenvolvedores

utilizem práticas de design centradas no usuário, como a eliminação de etapas desnecessárias e a simplificação dos processos. Aplicativos eficientes não só melhoram a experiência do usuário, mas também contribuem para a retenção de clientes e a promoção de interações repetidas (Bachmann).

- Satisfação do usuário: obtida por meio de feedback dos usuários e pesquisas, ajudando a entender como o público se sente ao usar a aplicação. É uma métrica subjetiva, mas de extrema relevância, pois reflete a percepção geral que os usuários têm em relação à aplicação. Essa satisfação pode ser medida por meio de pesquisas de feedback, onde os usuários avaliam sua experiência de uso em uma escala, ou por meio de métodos qualitativos, como entrevistas e grupos focais. Quando os usuários se sentem satisfeitos com a funcionalidade, a estética e a facilidade de uso da aplicação, é mais provável que se tornem defensores da marca, recomendando-a a outros. Portanto, entender os fatores que influenciam essa satisfação é crucial para o desenvolvimento contínuo e a melhoria da experiência do usuário, que deve ser sempre uma prioridade no processo de design e desenvolvimento (Nayebi).
- Taxa de erros: quantifica a frequência de erros que os usuários cometem e como isso afeta a interação geral. É uma métrica que fornece insights valiosos sobre onde os usuários podem encontrar dificuldades ao interagir com uma aplicação. Essa taxa se refere ao número de erros cometidos pelos usuários durante suas interações, seja por cliques incorretos, preenchimento de campos de forma errada ou falhas na execução de comandos. Um alto índice de erros pode indicar problemas na interface ou na usabilidade, que podem ser investigados e corrigidos através de testes de usabilidade. É essencial que os desenvolvedores se concentrem em reduzir essa taxa, implementando validações em tempo real, fornecendo feedback imediato e criando interfaces mais intuitivas. Reduzir a taxa de erros não apenas melhora a experiência do usuário, mas também aumenta a confiança do usuário na aplicação, contribuindo para uma interação mais positiva e eficaz (Bachmann).

Figura 4 – Métrica de taxa de erros

Tarefa	Quantidade de Erros	Taxa de Erro (%)
Login	15	3,0
Enviar Mensagem	25	5,0
Carregar Página	10	2,0
Salvar Dados	20	4,0
Buscar Informações	5	1,0
Atualizar Perfil	30	6,0

- **Tarefa**: Representa as ações que os usuários realizam no aplicativo texto do seu parágrafo.
- **Quantidade de Erros**: O número total de erros cometidos pelos usuários em cada tarefa.
- **Taxa de Erro (%)**: Percentual que representa a quantidade de erros em relação ao número total de tentativas de cada tarefa.

A Figura 4 apresenta uma análise abrangente da taxa de erro dos usuários em diversas tarefas de um aplicativo, sendo essencial para identificar áreas problemáticas que podem afetar a experiência do usuário. As colunas da tabela incluem, primeiramente, a “Tarefa”, que lista ações como Login, Enviar Mensagem, Carregar Página, Salvar Dados, Buscar Informação e Atualizar Perfil, cada uma representando uma funcionalidade chave do aplicativo. Em seguida, a coluna “Quantidade de Erros” fornece o número total de erros cometidos pelos usuários em cada tarefa, oferecendo uma visão quantitativa do desempenho e destacando quais atividades são mais desafiadoras, como a “Atualizar Perfil”, que registrou 30 erros. A coluna “Taxa de Erro (%)” expressa a proporção de erros em relação ao total de tentativas para cada tarefa. Mostra a gravidade dos erros, revelando, por exemplo, que “Enviar Mensagem” possui uma taxa de erro de 5%, indicando que esse percentual de tentativas resultou em falhas. A interpretação dos resultados mostra que a tarefa “Atualizar Perfil” não apenas tem a maior quantidade de erros, mas também a taxa de erro mais elevada (6%), sugerindo dificuldades significativas dos usuários nessa

funcionalidade, enquanto “Buscar Informação” apresenta a menor taxa de erro (1%), indicando um desempenho satisfatório.

Essas informações são exemplos sobre como as métricas são valiosas para desenvolvedores e designers, pois ajudam a priorizar melhorias e otimizações em áreas específicas, visando aumentar a satisfação do usuário e a eficácia geral do aplicativo.

TEMA 2 – MÉTRICAS PARA CÓDIGO-FONTE

A análise de métricas no código-fonte é fundamental para garantir a qualidade e a facilidade de manutenção dos sistemas de software. Essas métricas oferecem uma visão clara da estrutura e organização do código, destacando aspectos como complexidade, legibilidade e eficiência. Entre elas, estão a complexidade ciclomática, a coesão e o acoplamento, que ajudam a avaliar a saúde do código. Com essas informações, desenvolvedores e gestores podem tomar decisões mais acertadas sobre melhorias e manutenções.

A complexidade ciclomática, por exemplo, mede o número de caminhos independentes em um programa, sendo uma ferramenta poderosa para identificar áreas que podem apresentar dificuldades de teste. Isso é particularmente relevante em sistemas grandes e complexos, onde a probabilidade de erro aumenta à medida que o número de caminhos possíveis se multiplica.

A coesão e o acoplamento são métricas que avaliam a relação entre diferentes módulos do software; um código bem projetado deve apresentar alta coesão, indicando que os componentes de um módulo estão intimamente relacionados em sua funcionalidade, enquanto o acoplamento deve ser baixo, refletindo a independência entre os módulos.

A contagem de linhas de código, por sua vez, é frequentemente utilizada como um indicador inicial de tamanho, mas não deve ser vista isoladamente como um reflexo da qualidade do código.

Essa análise meticulosa das métricas é essencial para a manutenção e evolução de sistemas, fornecendo *insights* sobre onde e como as melhorias podem ser implementadas para garantir uma base de código mais saudável e sustentável (Lorenz).

Uma visão mais completa da qualidade do código envolve o uso de várias métricas que avaliam não apenas a complexidade, mas também outros aspectos

que impactam a manutenção do software. É essencial interpretar essas métricas dentro do contexto do projeto. Por exemplo, um grande número de linhas de código pode indicar um projeto robusto, mas também pode sugerir uma complexidade desnecessária se a estrutura não for adequada. Ao combinar diferentes métricas, as equipes conseguem ter uma visão mais precisa da saúde do código, permitindo a identificação de problemas antes que se tornem críticos. Essa abordagem é crucial para garantir que o software funcione corretamente e seja fácil de manter e evoluir, especialmente em um ambiente onde as mudanças são constantes (Hossain). A Figura 5 exibe uma comparação de cinco módulos de software, analisando métricas como linhas de código, complexidade ciclomática e coesão.

Figura 5 – Métrica de linhas de código

Módulo	Total de Linhas de Código	Complexidade Ciclomática	Coesão (Escala de 1-10)	Observações
Módulo A	320	15	9	Ideal com alta coesão e complexidade moderada: fácil de manter e testar.
Módulo B	450	25	7	Gerenciável, mas com dependências que aumentam a complexidade; manutenção viável.
Módulo C	500	40	5	Difícil de manter devido à baixa coesão e alta complexidade; recomenda-se reavaliação.
Módulo D	375	20	8	Boa coesão, porém algumas dependências aumentam a complexidade; manutenção requer cautela.
Módulo E	425	35	4	Reestruturação necessária devido à baixa coesão e alta complexidade; complicações na manutenção.

Um estudo empírico que examina a aplicação de métricas em dois produtos de software iterativos revela as nuances e os desafios envolvidos na medição da qualidade do código. Ao focar em métricas como acoplamento e complexidade, o estudo destaca como essas medidas impactam diretamente a capacidade de manutenção dos sistemas. Os resultados mostraram que um aumento na complexidade do código não só torna mais difícil a identificação de *bugs*, mas também resulta em maior tempo e esforço para implementar alterações. O estudo indica que o acoplamento excessivo entre módulos pode levar a um dominó de problemas, onde uma alteração em um módulo afeta negativamente outros, dificultando a evolução do software.

Essas avaliações reforçam a importância de se manter o código limpo e organizado, adotando práticas de programação que minimizem a complexidade e maximizem a coesão. Os pesquisadores sugerem que uma abordagem proativa na aplicação de métricas, incluindo revisões regulares e automação na análise de código, pode levar a melhores resultados em termos de qualidade e eficiência do desenvolvimento (Basili).

TEMA 3 – MÉTRICAS PARA TESTES

Um dos tipos mais relevantes de métricas no contexto dos testes de software é a cobertura de código, que avalia a proporção de código executado durante a fase de testes. Essa métrica desempenha um papel crucial ao destacar segmentos do código que ainda não foram testados, possibilitando que os desenvolvedores foquem nessas áreas e aumentem a robustez do sistema. A cobertura de código contribui para assegurar que os testes abrangem amplamente o sistema, reduzindo a chance de falhas inesperadas após a entrega. Dentro dessa métrica, encontramos variações importantes que permitem diferentes visões sobre a qualidade e a cobertura do código.

3.1 Cobertura de Código

A cobertura de código é uma das métricas mais discutidas e utilizadas no campo dos testes de software. Ela mede a extensão em que o código-fonte é executado durante os testes, expressando essa relação em porcentagem. Existem diferentes tipos de cobertura, entre os quais se destacam:

1. Cobertura de instruções: representa o percentual de linhas de código que foram executadas nos testes em relação ao total existente. Essa métrica é valiosa para identificar se cada instrução foi devidamente testada, reduzindo o risco de falhas em partes não verificadas do código.
2. Cobertura de *branches* (ou decisões): avalia quantas ramificações, como estruturas condicionais, foram testadas. Esse tipo de cobertura é essencial, pois erros podem surgir em caminhos específicos que os testes podem não ter coberto.
3. Cobertura de condições: essa extensão da cobertura de *branches* assegura que cada condição em uma instrução condicional seja avaliada tanto como verdadeira quanto como falsa, aumentando a precisão do teste.

Figura 6 – Métricas de teste: tipos de cobertura

Nome	Descrição	Representação
Cobertura de instruções	Mede a proporção de linhas de código executadas durante os testes, garantindo que cada linha seja testada pelo menos uma vez.	Barra de progresso com porcentagem de cobertura (ex: 80%)
Cobertura de Branches	Avalia se todos os caminhos de decisão (branches) do código foram testados, abrangendo diferentes fluxos de execução	Setas com caminhos testados marcados com ✓ e não testados com ✗.
Cobertura de Condições	Garante que cada condição em decisão foi avaliada tanto como verdadeira quanto como falsa	Duplas de caixas para cada condição (Verdadeiro/Falso)

A Figura 6 apresenta uma visão estruturada das principais métricas de cobertura de código, incluindo cobertura de instruções, *branches* e condições, ilustrando como cada métrica contribui para a qualidade e a completude dos testes.

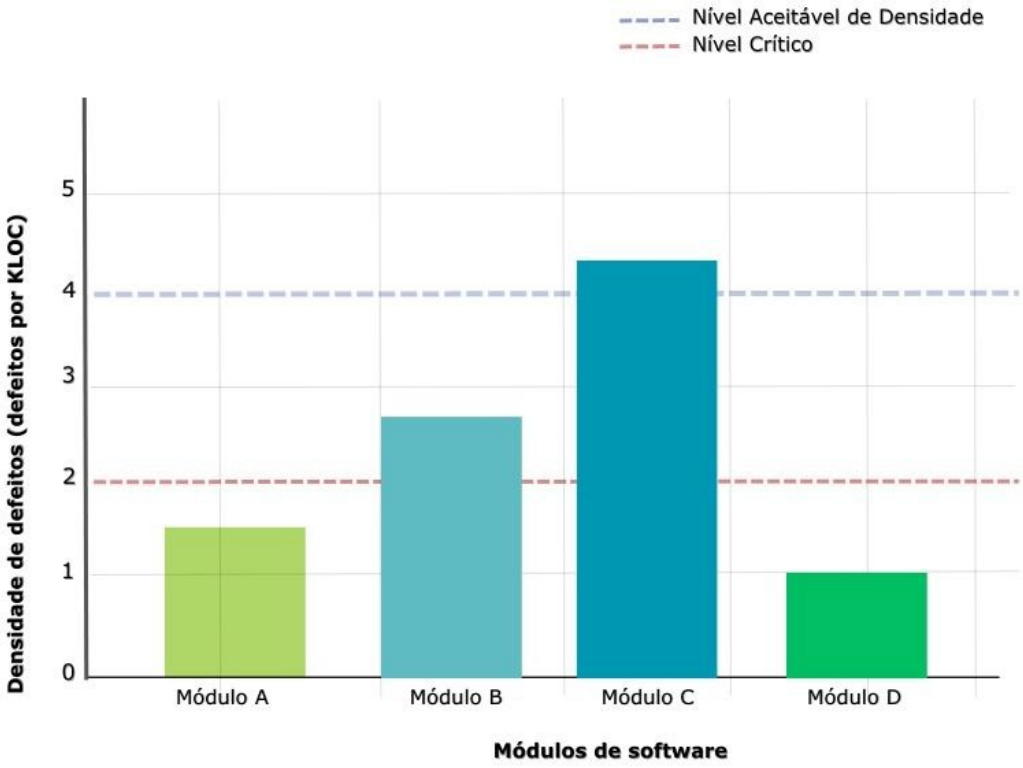
Ao implementar essas métricas, as equipes podem identificar partes do código que precisam de mais atenção, resultando em uma maior confiabilidade

do software. A análise profunda da cobertura pode reduzir significativamente o número de defeitos e aumentar a qualidade do produto final (Fraga).

3.2 Densidade de defeitos

A métrica de densidade de defeitos é fundamental para avaliar a qualidade do código, indicando o número de defeitos em um segmento específico do software, considerando seu tamanho. Essa métrica, usualmente medida em termos de defeitos por mil linhas de código (Kloc), permite que as equipes identifiquem áreas críticas que demandam atenção, seja para testes adicionais ou para uma possível reestruturação. A densidade de defeitos também é uma ferramenta útil para acompanhar o progresso da qualidade ao longo do tempo, possibilitando a comparação entre versões e a avaliação da prontidão do software para lançamento. Autores brasileiros destacam o valor desse acompanhamento contínuo para assegurar a qualidade em cada fase do desenvolvimento (Pereira).

Figura 7 – Métricas de teste: densidade de defeitos



A Figura 7 mostra a quantidade de defeitos por mil linhas de código (Kloc) em diferentes partes do software. Os módulos A, B, C e D estão no eixo horizontal, e a densidade de defeitos está no eixo vertical. As barras são

coloridas para indicar a qualidade: verde para baixa densidade de defeitos (Módulos A e D), amarelo para média (Módulo B) e vermelho para alta (Módulo C). As linhas pontilhadas mostram os padrões de qualidade, com um nível aceitável de 2 defeitos e um nível crítico de 4 defeitos. Isso ajuda a equipe a identificar onde precisam melhorar a qualidade do software antes de lançá-lo.

3.3 Tempo de teste

O tempo de teste é uma métrica que registra o tempo total despendido em diferentes atividades de teste, incluindo planejamento, execução e avaliação dos resultados. Compreender o tempo investido em cada fase do processo permite que as equipes:

- Identifiquem gargalos e áreas onde os testes podem ser otimizados;
- Avaliem a eficiência do processo de teste e façam ajustes conforme necessário.

A Figura 8 exemplifica várias fases a serem medidas.

Figura 8 – Métricas de teste: tempos

Crédito: Rosemari Pavan Rattmann/ChatGPT AI.

A análise do tempo de teste pode auxiliar na alocação de recursos, garantindo que as equipes estejam adequadamente equipadas para realizar os testes necessários dentro do cronograma estabelecido. O uso dessa métrica, como discutido por autores como Almeida e Ribeiro, demonstra a importância da gestão do tempo em projetos de desenvolvimento de software (Almeida).

3.4 Rastreamento de requisitos

O rastreamento de requisitos é uma métrica que analisa a conexão entre os requisitos de um software e os testes executados para sua validação. Essa métrica é fundamental para garantir que todos os requisitos definidos sejam considerados nos testes, prevenindo a implementação de funcionalidades que não atendem às necessidades dos usuários. Geralmente, o rastreamento é ilustrado por meio de gráficos ou tabelas, que indicam quais requisitos são cobertos por quais casos de teste.

O rastreamento de requisitos também ajuda a garantir a conformidade com normas e regulamentos, especialmente em setores como saúde e finanças, onde a qualidade do software é crítica. Pesquisadores brasileiros como Silva e Costa enfatizam que o rastreamento eficaz pode levar a um aumento significativo na qualidade do produto, pois assegura que cada requisito é testado de forma adequada (Silva).

A Figura 9 mostra um exemplo de rastreamento de requisitos em relação aos testes de validação.

Figura 9 – Métricas de teste: rastreamento de requisitos

ID do Requisito	Descrição do Requisito	Prioridade	ID do Teste	Descrição do Teste	Status do Testes	Obs
REQ-001	O sistema deve permitir login de usuários	Alta	TESTE-001	Testar login com credenciais válidas	Passou	Funcionou conforme esperado
REQ-002	O sistema deve	Alta	TESTE-002	Testar registro de ocorrência	Passou	Registro foi salvo corretamente

	registrar as ocorrências					
REQ-003	O sistema deve permitir edição de dados	Média	TESTE-003	Testar edição de dados do usuário	Falhou	O sistema não salvou as alterações

3.5 Aplicações práticas em ambientes industriais

As métricas de teste vão além de conceitos teóricos; sua aplicação em contextos industriais é fundamental para aprimorar continuamente os processos de desenvolvimento. Um estudo de caso da IBM demonstrou como as métricas relacionadas à eficácia na detecção de defeitos influenciaram positivamente a qualidade do software. Através da análise desses dados, as equipes foram capazes de reconhecer padrões de falhas e avaliar a eficiência dos testes em várias etapas do ciclo de vida do software. Com a adoção de métricas sistemáticas, foi possível reduzir de forma significativa a quantidade de defeitos nos produtos lançados, resultando em um aumento da satisfação dos clientes e da confiabilidade do software. Esses resultados mostram a importância de implementar métricas robustas para garantir a excelência no desenvolvimento de software e a satisfação do usuário final, como evidenciado na experiência da IBM com suas práticas de teste de software (IBM).

A aplicação de métricas oferece uma fundação robusta para a realização de decisões bem fundamentadas. Por exemplo, ao examinar a cobertura do código e a frequência de defeitos, as equipes podem avaliar se um software está preparado para lançamento ou se são necessários testes adicionais. Essa estratégia orientada por dados favorece uma alocação mais eficaz de recursos e esforços, assegurando que as áreas mais críticas recebam a atenção devida.

Alinhar as métricas aos objetivos do projeto é fundamental para maximizar o impacto das atividades de teste e promover a melhoria contínua no desenvolvimento de software. As métricas de teste devem estar conectadas às metas de negócio e às expectativas dos usuários para agregar verdadeiro valor ao processo (Winterhalter).

TEMA 4 – MÉTRICAS PARA PROJETOS DE MANUTENÇÃO E MELHORIAS

Em um mundo onde a tecnologia avança rapidamente, a manutenção de software se torna um elemento essencial para garantir que os sistemas continuem a funcionar de maneira eficaz e a atender às expectativas dos usuários. A manutenção não é apenas uma tarefa reativa; ela desempenha um papel proativo em adaptar e melhorar as aplicações, assegurando que permaneçam competitivas e funcionais em um mercado em constante mudança.

Para entender melhor como medir a eficácia dessas manutenções, é fundamental considerar um conjunto de métricas que avaliem diferentes aspectos do software, como sua qualidade, manutenibilidade e a eficiência dos processos envolvidos. Essa abordagem nos permitirá não apenas otimizar a manutenção, mas também maximizar o valor que o software oferece aos seus usuários.

4.1 Introdução à manutenção de software

A manutenção de software abrange uma série de atividades essenciais para garantir que um sistema continue a funcionar de maneira eficaz após sua implementação. Isso inclui corrigir falhas, aprimorar o desempenho e adaptar o software a novas condições ou requisitos. Surpreendentemente, essa fase muitas vezes requer mais recursos do que o próprio desenvolvimento inicial, tornando a adoção de métricas apropriadas crucial para avaliar a eficiência desses processos.

Um trabalho relevante nesse campo é o livro *Software Maintenance Success Recipes*, de Donald J. Reifer (2000), que oferece uma análise aprofundada das melhores práticas e métricas voltadas para a manutenção de software. Reifer enfatiza que o sucesso nesta área está intimamente ligado à habilidade de medir e acompanhar o desempenho do software ao longo do tempo. Essa abordagem permite que as equipes gerenciem proativamente as melhorias, garantindo que o software evolua conforme as necessidades dos usuários e as exigências do mercado.

4.2 Métricas para medir a manutenibilidade

A manutenibilidade é uma característica essencial em qualquer software, pois diz respeito à facilidade com que um sistema pode ser alterado para corrigir falhas ou aprimorar suas funcionalidades. Diversas métricas podem ser utilizadas para avaliar a manutenibilidade de um software (Bharat):

- Tempo de resolução de defeitos: mede quanto tempo leva para corrigir um defeito uma vez identificado. Um tempo mais curto indica uma maior manutenibilidade.
- Número de linhas de código por defeito: esta métrica ajuda a identificar a densidade de defeitos em relação ao código, permitindo avaliar a qualidade do código e sua complexidade.
- Cobertura de testes: refere-se à porcentagem do código que é testada por casos de teste. Alta cobertura geralmente está associada a uma maior manutenibilidade, pois os testes ajudam a garantir que as mudanças não introduzam novos defeitos.

Essas métricas são essenciais para identificar áreas de melhoria e monitorar o desempenho do software ao longo do tempo.

Figura 10 – Métricas de manutenibilidade

Métrica	Descrição	Valor Exemplar
Tempo de Resolução de Defeitos	Tempo necessário para corrigir um defeito após sua identificação. Um valor menor indica melhor manutenibilidade	5 horas
Número de Linhas de Código por Defeito	Densidade de defeitos rem relação ao total de linhas de código. Um número menor indica um código de melhor qualidade	20 linhas/código por defeito
Cobertura de Testes	Porcentagem do código que é coberta por testes automatizados. uma alta cobertura sugere maior manutenibilidade	85%

A Figura 10 exibe uma tabela que pode ser utilizada para comunicar rapidamente o estado das métricas de manutenibilidade para a equipe de desenvolvimento ou partes interessadas.

4.3 Métricas de qualidade durante a manutenção

A manutenção de software é um processo crítico que demanda atenção constante às métricas de qualidade. Algumas das métricas mais relevantes que ajudam a avaliar essa qualidade incluem:

- Complexidade ciclomática: mede a complexidade do código com base no número de caminhos independentes. Códigos mais complexos tendem a ser mais difíceis de manter; portanto, uma menor complexidade é desejável.
- Taxa de reabertura de defeitos: avalia quantos defeitos previamente fechados são reabertos. Uma alta taxa pode indicar problemas na eficácia da correção.
- Taxa de mudança: mede a frequência com que o código é alterado. Mudanças frequentes podem aumentar a probabilidade de novos defeitos; por isso, é importante monitorar essa métrica.

Figura 11 – Complexidade ciclomática

Condição	Ação
Se há pizza	Você escolhe pizza
Se não há pizza, mas há sanduíches	Você escolhe um sanduíche
Se não há pizza nem sanduíches, mas há frutas	Você escolhe frutas
Se não há nada disse	Você não come nada

A Figura 11 exemplifica como as decisões de escolha de um lanche podem ser influenciadas pelas opções disponíveis, refletindo a complexidade ciclomática do processo de decisão. Quanto mais opções houver, maior será a complexidade ciclomática, pois cada alternativa representa um caminho diferente na lógica de decisão.

Essas métricas ajudam a manter a qualidade do software, identificando tendências que podem comprometer a manutenibilidade.

4.4 Padrões de qualidade: ISO/IEC 25010

O padrão ISO/IEC 25010:2011, conhecido como SQuaRE (Requisitos e Avaliação de Qualidade de Sistemas e Software), oferece uma estrutura abrangente para a avaliação da qualidade do software. Ele abrange diversas características que são essenciais para medir e garantir a qualidade ao longo de todo o ciclo de vida do software. Um dos aspectos mais significativos deste padrão é a ênfase nas características relacionadas à manutenção, que são fundamentais para assegurar que os sistemas possam ser facilmente modificados, corrigidos e aprimorados com o tempo. As métricas desse padrão incluem:

- Analisabilidade: tempo médio para identificar a causa raiz de defeitos e a quantidade de documentação disponível.
- Modificabilidade: tempo necessário para implementar mudanças e a taxa de reuso de componentes.
- Testabilidade: percentual de cobertura de testes automatizados e a eficácia na detecção de falhas durante os testes.
- Estabilidade: taxa de falhas após modificações e o tempo médio entre falhas (MTBF).
- Substituibilidade: facilidade de troca de componentes e o tempo necessário para realizar substituições sem impactar o sistema.

O padrão ISO/IEC 25010 é fundamental para a definição e avaliação de métricas de qualidade em software, pois estabelece um *framework* que ajuda a garantir que os produtos atendam a requisitos específicos de qualidade, como confiabilidade, eficiência e manutenibilidade. Ao adotar essas normas, as organizações conseguem não apenas melhorar a qualidade do software, mas também facilitar a comunicação entre as partes interessadas e promover um entendimento comum sobre o que constitui um software de qualidade. A aplicação de padrões internacionais permite um alinhamento com as melhores práticas globais, contribuindo para o desenvolvimento de software mais robusto e sustentável. Além disso, a utilização de métricas padronizadas proporciona uma base para a tomada de decisões informadas e para a melhoria contínua dos

processos de desenvolvimento, aumentando a competitividade das empresas no mercado (Santos).

4.5 Exemplos práticos de métricas

As métricas desempenham um papel vital na melhoria da manutenção de software, sendo aplicadas de diversas maneiras para otimizar processos. Por exemplo, uma equipe pode utilizar o tempo de resolução de defeitos como um indicador de desempenho, estabelecendo metas para reduzir esse tempo a cada ciclo de manutenção. Se perceberem que o tempo médio para corrigir defeitos está aumentando, isso pode sinalizar a necessidade de revisar os processos de desenvolvimento ou a qualidade do código.

Monitorar a complexidade ciclomática do código também é uma prática importante. Ao acompanhar essa métrica ao longo do tempo, as equipes podem identificar se a manutenção se torna mais desafiadora e, assim, optar por refatorar o código para simplificá-lo. Além disso, utilizar a cobertura de testes como métrica assegura a qualidade do software, proporcionando confiança nas alterações realizadas. Isso permite que os desenvolvedores implementem melhorias com mais segurança.

As métricas relacionadas à manutenção e melhoria de software são ferramentas essenciais para assegurar a eficácia e a eficiência do processo de manutenção. Medindo aspectos como manutenibilidade, complexidade e tempo de resolução de defeitos, as equipes conseguem identificar áreas que precisam de aprimoramento e monitorar o desempenho ao longo do tempo, garantindo que o software atenda às expectativas dos usuários. A adoção de padrões como o ISO/IEC 25010 oferece uma estrutura robusta para a avaliação e melhoria da qualidade do software, assegurando que os sistemas permaneçam relevantes e funcionais em um cenário tecnológico em constante evolução.

Na gestão de projetos de software, as métricas são utilizadas para avaliar a eficiência e a eficácia em diferentes contextos, especialmente na comparação entre novos projetos e aqueles voltados para melhoria e manutenção. Em projetos iniciais, as métricas frequentemente se concentram na produtividade do desenvolvimento, na qualidade inicial e na conformidade com os requisitos, proporcionando uma visão clara do progresso e da qualidade desde o início. Esses aspectos são fundamentais para garantir que os objetivos de negócio sejam atingidos. Em contrapartida, em projetos de manutenção, as métricas se

enfocam em fatores como tempo de resolução de defeitos, densidade de defeitos, taxa de reabertura e complexidade ciclomática. Essas medições são cruciais para garantir a satisfação do cliente e a usabilidade do software existente, refletindo a natureza contínua e reativa desse tipo de trabalho. As diferenças nas métricas surgem de objetivos variados, contextos distintos e riscos específicos associados a cada tipo de projeto. Enquanto novos projetos lidam com incertezas e inovação, os de manutenção enfrentam o desafio de otimizar e corrigir softwares já estabelecidos. Compreender essas distinções permite que gerentes de projeto e equipes de desenvolvimento apliquem as métricas de maneira mais eficaz, promovendo a gestão da qualidade e a eficiência ao longo do ciclo de vida do software. Isso também contribui para a melhoria contínua, um fator essencial para a competitividade em um mercado tecnológico dinâmico.

A integração de diversas plataformas fornece uma base sólida para a exploração das métricas de software. Pesquisadores e profissionais podem se beneficiar do acesso a informações atualizadas e diversificadas, permitindo decisões mais informadas em seus projetos. O conhecimento obtido por meio dessas fontes pode resultar na implementação de melhores práticas, levando a melhorias significativas na qualidade do software. Com a crescente complexidade dos sistemas e as demandas do mercado, a adoção de métricas torna-se uma necessidade essencial para garantir que os produtos atendam aos padrões exigidos.

A Figura 12 apresenta, de forma clara, a distinção entre as métricas utilizadas em projetos novos e aqueles voltados para manutenção, ressaltando como cada cenário demanda abordagens distintas. Nos projetos novos, observa-se uma baixa taxa de defeitos, tempos de resolução de problemas mais rápidos, elevada satisfação do cliente e uma maior previsibilidade nos prazos de entrega, dado que a equipe está mais familiarizada com o código. Em contraste, projetos de manutenção frequentemente apresentam uma taxa de defeitos mais alta, com tempos de resolução que podem se estender devido à complexidade dos sistemas legados. A satisfação do cliente é monitorada de maneira contínua, refletindo as necessidades de ajustes e correções. Adicionalmente, a produtividade em projetos de manutenção é avaliada pela quantidade de correções implementadas, enquanto a complexidade do código e a cobertura de testes costumam ser mais desafiadoras. Os custos de implementação nesses

casos tendem a ser menos previsíveis, e a documentação frequentemente requer atualizações, o que evidencia a necessidade de estratégias específicas adaptadas a cada tipo de projeto.

Figura 12 – Comparação entre métricas

Métrica	Projetos Novos	Projetos de Manutenção
Taxa de Defeitos	Baixa durante o desenvolvimento inicial	Pode ser alta, especialmente em versões legadas
Tempo de Resolução de Defeitos	Geralmente menor devido a testes extensivos	Pode ser maior devido à complexidade do sistema
Satisfação do Cliente	Avaliada através de feedback inicial	Avaliada através de relatórios de usuários e revisões contínuas
Tempo de Entrega	Estimativas baseadas em planos de projeto	Frequentemente imprevisível devido a problemas planejados
Produtividade da Equipe	Medida por linhas de código ou funcionalidades entregues	Medida por alterações ou correções realizadas em um período
Complexidade do Código	Geralmente baixa, novo código	Alta, devido a dependências e interações com partes existentes
Cobertura de Testes	Alta cobertura é esperada	A cobertura pode ser menor, dependendo da documentação disponível
Custo de Implementação	Planejado de forma detalhada	Pode ser maior e menos previsível, devido a requisitos não especificados
Documentação	Criada durante o desenvolvimento	Muitas vezes incompleta ou desatualizada
Feedback de Performance	Avaliado após o lançamento	Monitorado continuamente e ajustado conforme necessário

TEMA 5 – REPOSITÓRIOS DE MÉTRICAS E FERRAMENTAS

As métricas de software são fundamentais para garantir a qualidade e a eficiência no desenvolvimento, manutenção e aprimoramento de sistemas. Servem como um alicerce objetivo para a tomada de decisões, permitindo que as equipes identifiquem áreas que requerem atenção, avaliem o desempenho de processos e produtos e promovam melhorias contínuas. Em um cenário

competitivo que demanda agilidade e qualidade, a adoção de métricas se torna uma prática essencial.

Para acessar informações e pesquisas sobre métricas de software, existem várias fontes valiosas, incluindo bibliotecas digitais e bases de dados acadêmicas. A IEEE Xplore Digital Library é uma referência importante, oferecendo uma vasta coleção de artigos acadêmicos, conferências e periódicos revisados por pares. Os documentos disponíveis nesta plataforma abrangem tópicos que vão desde métricas de qualidade até técnicas de teste e manutenção, proporcionando aos pesquisadores e profissionais do setor acesso a tendências atuais e práticas recomendadas. Além disso, a biblioteca contém estudos de caso que ilustram a aplicação prática de métricas em contextos reais, ajudando a transformar conceitos teóricos em implementações concretas (IEEE).

A Scopus também se destaca como uma plataforma abrangente, oferecendo acesso a artigos, resumos e citações sobre métricas de software. Conhecida pela qualidade de suas publicações, a Scopus permite que os pesquisadores identifiquem tendências em métricas de software e realizem análises de citações, monitorando o progresso de pesquisas ao longo do tempo. Com sua interface intuitiva e opções avançadas de busca, a Scopus é uma ferramenta poderosa para aqueles que desejam aprofundar seu entendimento sobre métricas aplicáveis ao desenvolvimento de software (Scopus).

5.1 Ferramentas para métricas de software

Existem várias ferramentas disponíveis no mercado para métricas de software, tanto pagas quanto gratuitas; várias delas são bastante importantes para avaliação da qualidade, confiabilidade, segurança e desempenho de software. Aqui estão algumas delas (Vagner):

- SonarQube: uma plataforma *open-source* amplamente utilizada para a análise contínua da qualidade do código e fornece uma gama de métricas essenciais, incluindo complexidade ciclomática, duplicação de código e cobertura de testes. Essa ferramenta permite que as equipes identifiquem rapidamente áreas problemáticas no código, ajudando a melhorar a manutenção e a legibilidade do software. Garante que a qualidade do código seja monitorada em cada etapa do desenvolvimento. Com relatórios detalhados e um painel de controle intuitivo, os desenvolvedores podem visualizar a evolução da qualidade do código ao longo do tempo,

o que facilita a priorização de refatorações e a implementação de boas práticas de desenvolvimento (Sonarqube).

- JUnit: é uma das ferramentas mais populares para a realização de testes unitários em Java. Ele não só permite a criação e execução de testes, mas também fornece métricas valiosas sobre a cobertura do código e a eficácia dos testes. Com o uso de anotações e um *framework* bem estruturado, o JUnit torna o processo de teste mais simples e eficiente. A cobertura de código gerada pelo JUnit ajuda a identificar partes do código que não foram testadas, possibilitando que os desenvolvedores se concentrem em áreas que necessitam de mais atenção. Além disso, a integração do JUnit com outras ferramentas, como o Maven e o Jenkins, facilita a automação dos testes, promovendo uma cultura de testes contínuos dentro das equipes de desenvolvimento.
- JMeter: é uma ferramenta de teste de desempenho que se destaca na medição da carga e eficiência de aplicações web. Ele é capaz de simular múltiplos usuários e medir a resposta do sistema sob diferentes condições de carga, permitindo que as equipes identifiquem gargalos de desempenho antes do lançamento de um produto. O JMeter oferece relatórios detalhados e gráficos que mostram a performance da aplicação durante os testes, o que é crucial para otimizar a infraestrutura e garantir que o sistema possa suportar um número elevado de acessos simultâneos. Além disso, sua interface amigável e suporte a múltiplos protocolos, como HTTP, FTP e JDBC, fazem do JMeter uma escolha versátil para testes de carga em diferentes tipos de aplicações.
- Postman: é uma ferramenta amplamente utilizada para testes de APIs, oferecendo um ambiente robusto para a criação, teste e documentação de chamadas de API. Ele permite que os desenvolvedores verifiquem o desempenho e a confiabilidade das interfaces, fornecendo métricas que ajudam a identificar problemas de resposta e a eficiência das chamadas. O Postman é especialmente útil na criação de conjuntos de testes automatizados, que podem ser integrados em *pipelines* de CI/CD. Além disso, sua capacidade de gerar documentação interativa a partir de coleções de testes facilita a colaboração entre equipes e a comunicação com *stakeholders*, assegurando que todos tenham acesso às informações sobre como as APIs funcionam.

- Checkmarx: é uma ferramenta especializada em segurança de software que realiza a análise estática de código para detectar vulnerabilidades potenciais. Ao integrar-se ao ciclo de desenvolvimento, o Checkmarx permite que as equipes identifiquem e mitiguem riscos de segurança logo no início do processo, antes que o código chegue à produção. Ele oferece relatórios detalhados que não apenas identificam falhas de segurança, mas também sugerem remediações específicas, promovendo uma abordagem proativa em relação à segurança de software. Com a crescente preocupação sobre segurança cibernética, o uso de ferramentas como o Checkmarx se tornou uma prática essencial para garantir que as aplicações sejam robustas e confiáveis.

A combinação de fundamentos teóricos com ferramentas práticas delineia um caminho eficaz para as equipes que buscam não apenas desenvolver, mas também manter sistemas de software de alta performance. As métricas oferecem uma perspectiva clara sobre a condição do código, possibilitando que as equipes foquem nas áreas que demandam maior atenção. Ao integrar a análise de métricas e soluções automatizadas ao ciclo de desenvolvimento, as equipes têm a oportunidade de criar um ambiente mais saudável e produtivo, essencial para o progresso de projetos em um mercado em constante transformação.

Com o aumento da complexidade dos sistemas de software e a crescente exigência por qualidade nos produtos, a utilização de métricas eficazes se torna cada vez mais crítica. Essas métricas desempenham um papel fundamental na avaliação e no aprimoramento dos processos de teste, além de contribuírem significativamente para a análise da qualidade dos resultados. Por isso, é vital que as equipes se familiarizem com as diversas métricas disponíveis e suas aplicações práticas, garantindo assim a excelência no desenvolvimento de software e a entrega de soluções que atendam às demandas do mercado.

FINALIZANDO

Durante nossa discussão, exploramos diversos aspectos cruciais que contribuem para a qualidade e a eficiência no desenvolvimento e manutenção de aplicações.

Iniciamos nossa jornada examinando as métricas aplicadas a *webapps* e aplicativos móveis, onde discutimos a importância de avaliar o desempenho, a segurança e a usabilidade. Esses indicadores são fundamentais para assegurar

que nossos produtos sejam não apenas funcionais, mas também seguros e agradáveis para os usuários.

Em seguida, nos concentramos nas métricas relacionadas ao código-fonte, destacando a cobertura de código, a densidade de defeitos e o tempo de teste. Além disso, enfatizamos o papel do rastreamento de requisitos e como essas métricas podem ser aplicadas em ambientes industriais para garantir que as expectativas sejam atendidas de maneira eficaz.

Depois, exploramos as métricas voltadas para projetos de manutenção e melhorias. Discutimos a manutenibilidade e como as métricas de qualidade podem ser aplicadas durante o processo de manutenção, alinhando-nos a padrões reconhecidos, como a ISO, e oferecendo exemplos práticos para uma melhor compreensão.

Por fim, apresentamos ferramentas e repositórios que facilitam a coleta e análise de métricas, como SonarQube, JUnit, JMeter, Postman e Checkmarx. O uso dessas ferramentas é essencial para um gerenciamento mais eficiente e uma abordagem proativa na melhoria contínua de nossos projetos.

Agradeço a participação de todos e espero que a compreensão dessas métricas e ferramentas ajude a enriquecer suas práticas em desenvolvimento de software.

REFERÊNCIAS

ALMEIDA, J. S.; RIBEIRO, F. G. A importância da gestão do tempo em projetos de software. **Revista Brasileira de Software**, v. 12, n. 3, p. 45-58, 2016.

BACHMANN, F.; BASS, L.; MERSON, P. **Measuring the Quality of Web Applications**. 2007.

BASILI, V. R.; BRIAND, L. C.; MELO, W. L. An Empirical Study of Object-Oriented Metrics in Two Iterative Software Products. In: **Proceedings of the 18th International Conference on Software Engineering**. Berlin: IEEE Computer Society, 1996. p. 247-256.

BHARAT, P.; MOHAPATRA, P. **Metrics for measuring the maintainability of software**. 2015.

FRAGA, L. F.; SOUSA, R. M. Métricas de teste de software: um guia prático. **Anais do Congresso Brasileiro de Software**, v. 20, n. 1, p. 215-226, 2015.

HOSSAIN, I. N. et al. A Survey on Code Quality Metrics. **Journal of Software Engineering and Applications**, v. 13, n. 6, p. 258-275, 2020. DOI: 10.4236/jsea.2020.136014.

IBM. **Improving Software Quality with Metrics**. IBM Developer.

IEEE Xplore Digital Library. Disponível em: <https://ieeexplore.ieee.org>. Acesso em: 23 jan. 2025.

KAPPEL, G. et al. **Web Engineering**: The Discipline of Systematic Development of Web Applications. 2006.

KNOTT, D. **Mobile Application Testing**: A Beginner's Guide. 2015.

LORENZ, M.; KIDD, J. **Object-Oriented Metrics**: A Practical Guide. 1994.

NAYEBI, M.; DESHARNAIS, J.; ABRAN, A. **A Systematic Literature Review of Quality Attributes and Metrics for Mobile Applications**. 2012.

PEREIRA, A. C. Qualidade de software e métricas de teste: uma análise crítica. **Journal of Software Engineering**, v. 10, n. 2, p. 67-79, 2017.

SANTOS, J. C.; FERREIRA, F. A. **Qualidade de Software**: Teoria e Prática. Rio de Janeiro: Ciência Moderna, 2018.

SCOPUS. Disponível em: <https://www.scopus.com>. Acesso em: 23 jan. 2025.

SILVA, A. M.; CAVALCANTI, E. Métricas de manutenção de software: um estudo de caso. In: **Anais do Congresso Brasileiro de Engenharia de Software**, 2016. p. 101-108.

_____. COSTA, R. S. Rastreamento de requisitos e sua importância para a qualidade do software. **Revista de Engenharia e Tecnologia**, v. 15, n. 4, p. 98-110, 2018.

SONARQUBE. **SonarQube Documentation**. Disponível em: <https://docs.sonarqube.org>. Acesso em: 23 jan. 2025.

VAGNER, F. M.; DA SILVA, R. S.; ALMEIDA, R. S. Ferramentas de teste de software: uma revisão de literatura. **Revista Brasileira de Sistemas de Informação**, v. 17, n. 2, p. 23-37, 2021.

WINTERHALTER, R. **Métricas de teste**: o que são e como aplicá-las. 1. ed. São Paulo: Novatec, 2021.